

Manuale operativo NetworkX

Tecniche di Programmazione in Python - grafi, esplorazioni e modelli pronti

Obiettivo del documento

Avere a disposizione durante l'esame una raccolta di metodi NetworkX ricorrenti, spiegati in modo pratico: quando usarli, cosa restituiscono, e come inserirli nel Model del progetto MVC/DAO.

1. Mappa mentale: come scegliere il metodo giusto

Richiesta tipica nel testo	Metodo/approccio	Idea da ricordare
Creare grafo non orientato	<code>nx.Graph()</code>	Arco A-B unico, senza direzione.
Creare grafo orientato	<code>nx.DiGraph()</code>	Archi A->B e B->A distinti.
Numero nodi/archi	<code>number_of_nodes()</code> , <code>number_of_edges()</code>	Serve quasi sempre dopo <code>buildGraph</code> .
Vicini/adiacenti di un nodo	<code>G.neighbors(n)</code> oppure <code>G[n]</code>	<code>G[n][vicino][\"weight\"]</code> legge il peso.
Aeroporti/nodi raggiungibili	<code>nx.node_connected_component(G, n)</code>	Solo per grafi non orientati.
Esiste un cammino tra A e B?	<code>nx.has_path(G, A, B)</code>	Controllo veloce prima di cercare percorsi.
Cammino minimo in numero di archi	<code>nx.shortest_path(G, A, B)</code>	Senza <code>weight</code> : minimizza il numero di archi.
Cammino minimo pesato	<code>nx.shortest_path(G, A, B, weight=\"weight\")</code>	Con <code>weight</code> : minimizza la somma dei pesi.
Nodi entro k tratte	<code>nx.single_source_shortest_path_length(G, A, cutoff=k)</code>	BFS per livelli.
Tutti i cammini semplici entro k	<code>nx.all_simple_paths(G, A, B, cutoff=k)</code>	Non ripete nodi; può essere costoso.
Massimizzare un punteggio con vincoli	ricorsione/backtracking personalizzata	NetworkX aiuta, ma spesso serve DFS tua.
Componenti connesse	<code>nx.connected_components(G)</code>	Gruppi separati di nodi collegati.
Sottografo su alcuni nodi	<code>G.subgraph(lista_nodi).copy()</code>	Utile dopo filtri o componente maggiore.
Albero ricoprente minimo/massimo	<code>nx.minimum_spanning_tree / maximum_spanning_tree</code>	Quando chiede collegare tutti minimizzando/massimizzando pesi.

2. Setup base nel Model

Negli esercizi d'esame conviene tenere il grafo come attributo del Model, insieme a eventuali mappe ID -> oggetto.

```
import networkx as nx
from database.DAO import DAO

class Model:
    def __init__(self):
        self._graph = nx.Graph()          # oppure nx.DiGraph()
        self._idMap = {}                  # es: id aeroporto -> oggetto Airport
        self._bestPath = []               # utile per ricorsione
        self._bestScore = 0
```

Regola MVC/DAO

DAO = query SQL e recupero dati. Model = costruzione grafo e algoritmi. Controller = legge input dalla View e stampa risultati. View = solo interfaccia grafica.

3. Creazione e modifica del grafo

`nx.Graph()` / `nx.DiGraph()`

Quando usarlo	Quando il testo dice grafo non orientato usa <code>nx.Graph()</code> . Quando dice orientato/diretto usa <code>nx.DiGraph()</code> .
Cosa restituisce	Un oggetto grafo vuoto.
Attenzione	In un grafo non orientato A-B e B-A sono lo stesso arco. In un grafo orientato sono due archi diversi.

```
self._graph = nx.Graph()      # non orientato
self._graph = nx.DiGraph()    # orientato
```

`add_node()`, `add_nodes_from()`

Quando usarlo	Per aggiungere nodi singoli o una lista di oggetti/nodi.
Cosa restituisce	Modifica il grafo in-place, non restituisce una lista.
Attenzione	-

```
self._graph.add_node(aeroporto)
self._graph.add_nodes_from(lista_aeroporti)

# Tipico dopo una query DAO
nodi = DAO.getAirports(...)
self._graph.add_nodes_from(nodi)
```

`add_edge()`, `add_edges_from()`

Quando usarlo	Per aggiungere archi. Se il grafo e' pesato, salva il peso come attributo <code>weight</code> .
Cosa restituisce	Modifica il grafo in-place.
Attenzione	Se aggiungi due volte lo stesso arco in <code>nx.Graph()</code> , non crea un duplicato: aggiorna/sovrascrive gli attributi.

```
self._graph.add_edge(a1, a2)
self._graph.add_edge(a1, a2, weight=peso)

# Da righe DAO: (id1, id2, peso)
for id1, id2, peso in DAO.getEdges(...):
    n1 = self._idMap[id1]
```

```
n2 = self._idMap[id2]
self._graph.add_edge(n1, n2, weight=peso)
```

clear()

Quando usarlo	Prima di ricostruire il grafo con parametri diversi.
Cosa restituisce	Svuota nodi e archi.
Attenzione	-

```
def buildGraph(self, parametro):
    self._graph.clear()
    self._idMap.clear()
    # poi ricarico nodi e archi
```

4. Informazioni base sul grafo

number_of_nodes(), number_of_edges()

Quando usarlo	Subito dopo aver costruito il grafo, per stampare dimensioni richieste dal testo.
Cosa restituisce	Interi.
Attenzione	-

```
def getGraphDetails(self):
    return self._graph.number_of_nodes(), self._graph.number_of_edges()

# equivalente:
# len(self._graph.nodes), len(self._graph.edges)
```

nodes, edges

Quando usarlo	Per ottenere liste di nodi o archi, ad esempio da stampare o caricare nei dropdown.
Cosa restituisce	NodeView / EdgeView; spesso conviene convertirli in list.
Attenzione	-

```
nodi = list(self._graph.nodes)
archi = list(self._graph.edges)
archi_con_peso = list(self._graph.edges(data=True))

for u, v, data in self._graph.edges(data=True):
    print(u, v, data["weight"])
```

has_node(), has_edge()

Quando usarlo	Per verificare se un nodo/arco esiste prima di leggerlo.
Cosa restituisce	Booleano True/False.
Attenzione	-

```
if self._graph.has_node(a):
    print("Nodo presente")

if self._graph.has_edge(a1, a2):
    peso = self._graph[a1][a2]["weight"]
```

5. Vicini, adiacenze e grado

G.neighbors(n)

Quando usarlo	Quando il testo chiede nodi adiacenti, vicini, aeroporti collegati direttamente.
Cosa restituisce	Iteratore sui vicini del nodo.
Attenzione	Se il nodo non esiste nel grafo, NetworkX solleva errore.

	Meglio controllare <code>has_node</code> se hai dubbi.
--	--

```
def getNeighbors(self, nodo):
    return list(self._graph.neighbors(nodo))
```

G[n] / G.adj[n]

Quando usarlo	Quando vuoi leggere anche gli attributi degli archi verso i vicini.
Cosa restituisce	Dizionario-like: vicino -> dizionario attributi arco.
Attenzione	Questo e' spesso il modo piu' rapido per stampare vicini ordinati per peso.

```
def getNeighborsWithWeight(self, nodo):
    result = []
    for vicino, dati in self._graph[nodo].items():
        peso = dati["weight"]
        result.append((vicino, peso))
    result.sort(key=lambda x: x[1], reverse=True)
    return result
```

degree

Quando usarlo	Quando il testo chiede il grado, cioe' quanti archi incidenti ha un nodo. Con weight calcola grado pesato.
Cosa restituisce	Intero per un nodo, oppure vista/dizionario per piu' nodi.
Attenzione	degree senza weight conta archi. degree con weight somma i pesi degli archi incidenti.

```
grado = self._graph.degree[nodo]
grado_pesato = self._graph.degree(nodo, weight="weight")

# ordinare nodi per grado decrescente
classifica = sorted(
    self._graph.degree,
    key=lambda x: x[1],
    reverse=True
)
```

6. Connettivita'

nx.has_path(G, source, target)

Quando usarlo	Quando il testo chiede se due nodi sono collegati o se esiste un percorso.
Cosa restituisce	Booleano True/False.
Attenzione	-

```
def areConnected(self, a1, a2):
    return nx.has_path(self._graph, a1, a2)
```

nx.is_connected(G)

Quando usarlo	Quando chiede se tutto il grafo e' connesso.
Cosa restituisce	Booleano True/False.
Attenzione	Funziona sui grafi non orientati. Per grafi orientati esistono <code>is_strongly_connected</code> e <code>is_weakly_connected</code> .

```
def isGraphConnected(self):
    if self._graph.number_of_nodes() == 0:
        return False
    return nx.is_connected(self._graph)
```

`nx.connected_components(G)`

Quando usarlo	Quando chiede componenti connesse, gruppi separati, componente piu' grande.
Cosa restituisce	Generatore di set di nodi, uno per componente.
Attenzione	-

```
def getConnectedComponents(self):
    comps = list(nx.connected_components(self._graph))
    comps.sort(key=len, reverse=True)
    return comps

def getLargestComponent(self):
    comp = max(nx.connected_components(self._graph), key=len)
    return self._graph.subgraph(comp).copy()
```

`nx.node_connected_component(G, nodo)`

Quando usarlo	Quando chiede tutti i nodi raggiungibili da un nodo in un grafo non orientato.
Cosa restituisce	Set di nodi nella componente del nodo.
Attenzione	-

```
def getReachableNodes(self, nodo):
    return nx.node_connected_component(self._graph, nodo)
```

7. Esplorazioni DFS e BFS

Scelta veloce

BFS = ragionamento per livelli e distanza minima in numero di archi. DFS = esplorazione in profondita', cammini, backtracking, combinazioni.

`nx.dfs_edges(G, source)`

Quando usarlo	Quando vuoi vedere gli archi visitati da una DFS standard a partire da un nodo.
Cosa restituisce	Iteratore di tuple (padre, figlio).
Attenzione	L'ordine dipende dall'ordine interno dei vicini. Per ottimizzazioni con vincoli, meglio scrivere DFS ricorsiva personalizzata.

```
def getDfsEdges(self, source):
    return list(nx.dfs_edges(self._graph, source=source))
```

`nx.dfs_preorder_nodes(G, source)`

Quando usarlo	Quando vuoi l'ordine dei nodi visitati da DFS.
Cosa restituisce	Iteratore di nodi.
Attenzione	-

```
def getDfsNodes(self, source):
    return list(nx.dfs_preorder_nodes(self._graph, source=source))
```

`nx.bfs_edges(G, source)`

Quando usarlo	Quando vuoi gli archi visitati da BFS, cioe' prima vicini diretti, poi distanza 2, ecc.
Cosa restituisce	Iteratore di tuple (padre, figlio).
Attenzione	-

```
def getBfsEdges(self, source):
    return list(nx.bfs_edges(self._graph, source=source))
```

`nx.bfs_tree(G, source)`

Quando usarlo	Quando vuoi l'albero di visita BFS da un nodo.
Cosa restituisce	Un grafo orientato che rappresenta l'albero BFS.
Attenzione	-

```
def getBfsTreeNodes(self, source):
    tree = nx.bfs_tree(self._graph, source=source)
    return list(tree.nodes())
```

`nx.descendants_at_distance(G, source, distance)`

Quando usarlo	Quando chiede nodi esattamente a distanza k archi da un nodo.
Cosa restituisce	Set di nodi a distanza esatta.
Attenzione	Per distanza al massimo k usa <code>single_source_shortest_path_length</code> con cutoff.

```
def getNodesAtDistance(self, source, k):
    return nx.descendants_at_distance(self._graph, source, k)
```

8. Cammini minimi e distanze

`nx.shortest_path(G, source, target)`

Quando usarlo	Quando chiede il cammino minimo in numero di archi/tratte, senza considerare i pesi.
Cosa restituisce	Lista di nodi dal source al target.
Attenzione	Senza parametro weight, ogni arco vale 1.

```
def getShortestPath(self, a1, a2):
    if not nx.has_path(self._graph, a1, a2):
        return []
    return nx.shortest_path(self._graph, source=a1, target=a2)
```

`nx.shortest_path_length(G, source, target)`

Quando usarlo	Quando chiede la distanza minima in numero di archi.
Cosa restituisce	Intero.
Attenzione	-

```
def getDistance(self, a1, a2):
    if not nx.has_path(self._graph, a1, a2):
        return None
    return nx.shortest_path_length(self._graph, source=a1, target=a2)
```

`nx.shortest_path(..., weight="weight")`

Quando usarlo	Quando chiede cammino minimo pesato: minimizzare costo, distanza, tempo, prezzo.
Cosa restituisce	Lista di nodi.
Attenzione	Attenzione: minimizza la somma dei pesi. Se il peso rappresenta un beneficio da massimizzare, non usare direttamente <code>shortest_path</code> .

```
def getWeightedShortestPath(self, a1, a2):
    if not nx.has_path(self._graph, a1, a2):
        return []
    return nx.shortest_path(
        self._graph,
        source=a1,
        target=a2,
        weight="weight"
    )
```

`nx.dijkstra_path / nx.dijkstra_path_length`

Quando usarlo	Equivalente esplicito per cammini minimi pesati con pesi non negativi.
Cosa restituisce	Lista di nodi oppure valore numerico della distanza.
Attenzione	Dijkstra e' adatto quando i pesi sono costi positivi da minimizzare.

```
path = nx.dijkstra_path(self._graph, a1, a2, weight="weight")
dist = nx.dijkstra_path_length(self._graph, a1, a2, weight="weight")
```

`nx.single_source_shortest_path_length(G, source, cutoff=k)`

Quando usarlo	Quando chiede tutti i nodi raggiungibili entro k archi/tratte.
Cosa restituisce	Dizionario nodo -> distanza minima dal source.
Attenzione	Questo e' BFS per livelli gia' pronta.

```
def getReachableWithinK(self, source, k):
    distanze = nx.single_source_shortest_path_length(
        self._graph,
        source=source,
        cutoff=k
    )
    # escludo il source, se serve
    return [(n, d) for n, d in distanze.items() if n != source]
```

`nx.all_shortest_paths(G, source, target)`

Quando usarlo	Quando chiede tutti i cammini minimi, non solo uno.
Cosa restituisce	Generatore di liste di nodi.
Attenzione	-

```
def getAllShortestPaths(self, a1, a2):
    if not nx.has_path(self._graph, a1, a2):
        return []
    return list(nx.all_shortest_paths(self._graph, a1, a2))
```

9. Tutti i cammini semplici

`nx.all_simple_paths(G, source, target, cutoff=k)`

Quando usarlo	Quando il testo chiede tutti i percorsi/cammini senza ripetere nodi, eventualmente con massimo k archi.
Cosa restituisce	Generatore di liste di nodi.
Attenzione	Puo' esplodere in tempo/memoria: il numero di cammini semplici puo' essere enorme.

```
def getAllSimplePaths(self, a1, a2, k):
    if not nx.has_path(self._graph, a1, a2):
        return []
    return list(nx.all_simple_paths(
        self._graph,
        source=a1,
        target=a2,
        cutoff=k
    ))
```

Calcolare il peso di un path

Quando usarlo	Da usare insieme ad <code>all_simple_paths</code> o alla ricorsione.
Cosa restituisce	Numero: somma dei pesi degli archi del path.
Attenzione	-

```
def getPathWeight(self, path):
    tot = 0
    for i in range(len(path) - 1):
        u = path[i]
        v = path[i+1]
        tot += self._graph[u][v]["weight"]
    return tot
```

Massimo peso usando all_simple_paths

Quando usarlo	Alternativa rapida se il testo NON obbliga a scrivere ricorsione.
Cosa restituisce	Miglior cammino e relativo peso.
Attenzione	Se il testo richiede una funzione ricorsiva, usa il modello di backtracking nella sezione 14.

```
def getBestPathByAllSimplePaths(self, a1, a2, max_archi):
    best_path = []
    best_score = 0

    if not nx.has_path(self._graph, a1, a2):
        return best_path, best_score

    for path in nx.all_simple_paths(self._graph, a1, a2, cutoff=max_archi):
        score = self.getPathWeight(path)
        if score > best_score:
            best_score = score
            best_path = path

    return best_path, best_score
```

10. Grafi orientati: metodi da ricordare

successors(), predecessors()

Quando usarlo	In nx.DiGraph: successors = nodi raggiungibili con archi uscenti; predecessors = nodi che entrano nel nodo.
Cosa restituisce	Iteratori di nodi.
Attenzione	-

```
successori = list(self._graph.successors(nodo))
predecessori = list(self._graph.predecessors(nodo))

# Nota: in DiGraph, neighbors(nodo) equivale a successors(nodo)
```

in_degree, out_degree

Quando usarlo	Quando il testo distingue entranti/uscenti.
Cosa restituisce	Intero o vista di coppie nodo-grado.
Attenzione	-

```
entranti = self._graph.in_degree(nodo)
uscenti = self._graph.out_degree(nodo)

entranti_pesati = self._graph.in_degree(nodo, weight="weight")
uscenti_pesati = self._graph.out_degree(nodo, weight="weight")
```

strongly_connected_components / weakly_connected_components

Quando usarlo	Per componenti connesse in grafi orientati.
Cosa restituisce	Generatori di set di nodi.
Attenzione	Strong: rispetta la direzione in entrambi i versi. Weak: ignora temporaneamente la direzione.

```
forti = list(nx.strongly_connected_components(self._graph))
deboli = list(nx.weakly_connected_components(self._graph))
```

11. Sottografi e filtri

G.subgraph(nodes).copy()

Quando usarlo	Quando vuoi lavorare solo su un gruppo di nodi, per esempio la componente maggiore.
Cosa restituisce	Un nuovo grafo con quei nodi e gli archi tra loro.
Attenzione	Senza .copy() ottieni una vista collegata al grafo originale; per esame di solito conviene copy().

```
def buildSubgraphFromNodes(self, nodes):
    return self._graph.subgraph(nodes).copy()
```

remove_node(), remove_nodes_from()

Quando usarlo	Quando devi eliminare nodi non validi dopo una costruzione.
Cosa restituisce	Modifica il grafo.
Attenzione	-

```
self._graph.remove_node(nodo)
self._graph.remove_nodes_from(lista_nodi_da_eliminare)
```

remove_edge(), remove_edges_from()

Quando usarlo	Quando devi eliminare archi non validi.
Cosa restituisce	Modifica il grafo.
Attenzione	-

```
self._graph.remove_edge(a1, a2)
self._graph.remove_edges_from(lista_archi_da_eliminare)
```

12. Alberi ricoprenti e misure utili

nx.minimum_spanning_tree(G, weight="weight")

Quando usarlo	Quando chiede di collegare tutti i nodi minimizzando il costo totale.
Cosa restituisce	Un grafo albero ricoprente minimo.
Attenzione	Ha senso su grafo non orientato e connesso; se non connesso produce una foresta.

```
T = nx.minimum_spanning_tree(self._graph, weight="weight")
costo = T.size(weight="weight")
```

nx.maximum_spanning_tree(G, weight="weight")

Quando usarlo	Quando chiede di collegare tutti i nodi massimizzando il peso totale.
Cosa restituisce	Un grafo albero ricoprente massimo.
Attenzione	-

```
T = nx.maximum_spanning_tree(self._graph, weight="weight")
peso_totale = T.size(weight="weight")
```

nx.degree_centrality(G)

Quando usarlo	Raramente ma utile se chiede nodi piu' centrali/importanti per numero di collegamenti.
Cosa restituisce	Dizionario nodo -> centralita'.

Attenzione

-

```
centralita = nx.degree_centrality(self._graph)
classifica = sorted(centralita.items(), key=lambda x: x[1], reverse=True)
```

13. Modelli pronti per il Model

13.1 Build graph da DAO con nodi oggetto e archi pesati

```
def buildGraph(self, parametro):
    self._graph.clear()
    self._idMap.clear()

    nodi = DAO.getNodes(parametro)
    self._graph.add_nodes_from(nodi)

    self._idMap = {n.ID: n for n in nodi}

    edges = DAO.getEdges(self._idMap.keys()) # righe: id1, id2, peso
    for id1, id2, peso in edges:
        n1 = self._idMap[id1]
        n2 = self._idMap[id2]
        self._graph.add_edge(n1, n2, weight=peso)

    return self._graph.number_of_nodes(), self._graph.number_of_edges()
```

13.2 Vicini ordinati per peso decrescente

```
def getNeighborsSortedByWeight(self, nodo):
    result = []
    for vicino, dati in self._graph[nodo].items():
        result.append((vicino, dati["weight"]))

    result.sort(key=lambda x: x[1], reverse=True)
    return result
```

13.3 Nodi raggiungibili entro k passi

```
def getReachableWithinK(self, source, k):
    distanze = nx.single_source_shortest_path_length(
        self._graph,
        source=source,
        cutoff=k
    )
    result = []
    for nodo, distanza in distanze.items():
        if nodo != source:
            result.append((nodo, distanza))
    result.sort(key=lambda x: x[1])
    return result
```

13.4 Cammino minimo e stampa con pesi arco per arco

```
def getShortestPathWithWeights(self, a1, a2):
    if not nx.has_path(self._graph, a1, a2):
        return [], 0

    path = nx.shortest_path(self._graph, a1, a2)
    dettagli = []
    totale = 0

    for i in range(len(path) - 1):
```

```

        u = path[i]
        v = path[i+1]
        peso = self._graph[u][v]["weight"]
        dettagli.append((u, v, peso))
        totale += peso

    return dettagli, totale

```

13.5 Componente maggiore e nodi ordinati per grado

```

def getLargestComponentNodesByDegree(self):
    comp = max(nx.connected_components(self._graph), key=len)
    sub = self._graph.subgraph(comp).copy()

    result = list(sub.degree)
    result.sort(key=lambda x: x[1], reverse=True)
    return result

```

14. Quando NetworkX non basta: ricorsione/backtracking

Quando usarla

Usa ricorsione quando il testo chiede il percorso migliore con vincoli personalizzati: massimo t tratte, senza ripetere nodi, punteggio da massimizzare/minimizzare, regole particolari sui nodi o sugli archi.

14.1 Schema generale di backtracking su grafo

```

def cerca(self, start, end, max_tratte):
    self._bestPath = []
    self._bestScore = 0

    parziale = [start]
    self._ricorsione(parziale, end, max_tratte, 0)

    return self._bestPath, self._bestScore

def _ricorsione(self, parziale, end, max_tratte, score_attuale):
    ultimo = parziale[-1]

    # Caso 1: sono arrivato alla destinazione
    if ultimo == end:
        if score_attuale > self._bestScore:
            self._bestScore = score_attuale
            self._bestPath = list(parziale)
        return

    # Caso 2: ho già usato il numero massimo di archi
    if len(parziale) - 1 == max_tratte:
        return

    # Espansione DFS
    for vicino in self._graph.neighbors(ultimo):
        if vicino not in parziale: # evita cicli e ripetizioni
            peso = self._graph[ultimo][vicino]["weight"]

            parziale.append(vicino)
            self._ricorsione(parziale, end, max_tratte, score_attuale + peso)
            parziale.pop()

```

14.2 Varianti rapide della ricorsione

Obiettivo	Modifica da fare
Minimizzare invece di massimizzare	Inizializza bestScore a un valore molto alto e aggiorna se <code>score_attuale < bestScore</code> .
Accettare solo path esattamente di k archi	Aggiorna best solo se <code>ultimo == end</code> e <code>len(parziale)-1 == k</code> .
Permettere massimo k archi	Aggiorna quando arrivi a end, anche se hai usato meno di k archi.
Evitare nodi con certe proprietà	Prima di append controlla la proprietà del vicino.
Evitare archi con peso basso	Prima di append controlla <code>if peso >= soglia</code> .
Contare numero di soluzioni	Invece di best, incrementa un contatore quando trovi una soluzione valida.

15. Mini-modelli Controller per usare i metodi

15.1 Dopo aver costruito il grafo

```
def handleCreaGrafo(self, e):
    try:
        x = int(self._view.txt_x.value)
    except ValueError:
        self._view.create_alert("Inserire un numero valido")
        return

    nNodes, nEdges = self._model.buildGraph(x)
    self._view.txt_result.controls.append(
        ft.Text(f"Grafo creato: {nNodes} nodi, {nEdges} archi")
    )

    nodi = self._model.getNodes()
    self._view.dd_start.options = [ft.dropdown.Option(key=str(n.ID), text=str(n)) for n in nodi]
    self._view.dd_end.options = [ft.dropdown.Option(key=str(n.ID), text=str(n)) for n in nodi]
    self._view.update_page()
```

15.2 Chiamare una ricerca ricorsiva

```
def handleCerca(self, e):
    start = self._model.getNodeById(int(self._view.dd_start.value))
    end = self._model.getNodeById(int(self._view.dd_end.value))
    t = int(self._view.txt_t.value)

    path, score = self._model.cerca(start, end, t)

    if len(path) == 0:
        self._view.txt_result.controls.append(ft.Text("Nessun percorso trovato"))
    else:
        self._view.txt_result.controls.append(ft.Text(f"Score migliore: {score}"))
        for i in range(len(path)-1):
            peso = self._model.getEdgeWeight(path[i], path[i+1])
            self._view.txt_result.controls.append(ft.Text(f"{path[i]} -> {path[i+1]} | peso {peso}"))

    self._view.update_page()
```

16. Checklist finale da usare all'esame

- Capire se il grafo e' orientato o non orientato: Graph o DiGraph.

- Capire cosa sono i nodi: entita' della traccia, spesso oggetti del model.
- Capire cosa sono gli archi: relazione tra due nodi, spesso query GROUP BY nel DAO.
- Capire se il peso e' un costo da minimizzare o un beneficio da massimizzare.
- Costruire il grafo nel Model, non nel Controller e non nel DAO.
- Usare DAO solo per recuperare dati dal database.
- Dopo buildGraph stampare sempre number_of_nodes e number_of_edges.
- Per vicini ordinati: `G[n].items()` + peso + sort.
- Per connessione: `nx.has_path`.
- Per minimo numero di archi: `shortest_path` senza weight.
- Per minimo costo pesato: `shortest_path` con `weight="weight"`.
- Per entro k passi: `single_source_shortest_path_length` con cutoff.
- Per tutti i cammini semplici: `all_simple_paths` con cutoff, ma attenzione al costo.
- Per miglior percorso con vincoli: ricorsione/backtracking.
- Nella ricorsione ricordare sempre append, chiamata ricorsiva, pop.

17. Riferimenti ufficiali NetworkX

- NetworkX Tutorial: <https://networkx.org/documentation/stable/tutorial.html>
- Traversal algorithms: <https://networkx.org/documentation/stable/reference/algorithms/traversal.html>
- Shortest paths: https://networkx.org/documentation/stable/reference/algorithms/shortest_paths.html
- all_simple_paths: https://networkx.org/documentation/stable/reference/algorithms/generated/networkx.algorithms.simple_paths.all_simple_paths.html
- connected_components: https://networkx.org/documentation/stable/reference/algorithms/generated/networkx.algorithms.components.connected_components.html
- Graph.neighbors: <https://networkx.org/documentation/stable/reference/classes/generated/networkx.Graph.neighbors.html>
- Graph.degree: <https://networkx.org/documentation/stable/reference/classes/generated/networkx.Graph.degree.html>
- Graph.subgraph: <https://networkx.org/documentation/stable/reference/classes/generated/networkx.Graph.subgraph.html>